

Two-Tier System — Complete User Manual

This manual explains, in plain language and one small step at a time, how to build the whole project from nothing: a private banking assistant that answers most questions itself on your own computer, and only asks a paid AI (Claude) for help with the harder questions.

You do not need to know how to code. Every step tells you exactly what to type or click, and what you should see afterwards so you know it worked.

How to use this manual: work through it from top to bottom, in order. Do not skip ahead. Each part depends on the one before it.

Contents

1. [Words this manual uses](#)
2. [What you will end up with](#)
3. [Before you start](#)
4. [Install the basic tools](#) — Ollama, Python, Git
5. [Get the project files onto your computer](#) — clone from GitHub, or set it up for the first time
6. [Set up the project's Python environment](#)
7. [Get a Claude API key](#)
8. [Download the two AI models](#)
9. [Create the practice \(synthetic\) data](#)
10. [Run the system for the first time](#)
11. [Set up and open the dashboard](#)
12. [\(Optional\) Train your own custom models on Google Colab](#)
13. [Bring your trained models into Ollama](#)
14. [Everyday commands](#)
15. [If something goes wrong](#)

0. Words this manual uses

A few words come up again and again. Here is what each one means, in plain terms.

Word	What it means
Terminal (Windows: "PowerShell")	A plain black or white window where you type commands instead of clicking buttons. It is a normal, built-in part of every computer — it is not something you need to download.
Command	A line of text you type into the terminal, then press Enter to run it. This manual shows every command in a box like this: <code>like this</code>
Folder / Directory	Same thing — a place on your computer where files are kept, like a folder on your Desktop.

Word	What it means
Repository (or "repo")	A folder whose history is tracked by a tool called Git, so changes can be saved, shared, and undone. Our whole project lives in one repository.
GitHub	A website that stores a copy of a repository online, so it can be shared with other people or other computers.
Clone	Downloading a full copy of a repository from GitHub onto your own computer.
Virtual environment (venv)	A private, self-contained box for this one project's Python software, so it never clashes with anything else on your computer.
API key	A long, secret password-like code that lets our program talk to Claude (an AI made by Anthropic) over the internet. It is tied to a billing account, so it must be kept private.
Model	The actual AI "brain" that reads a question and writes an answer. This project uses three: two small ones that run on your own computer (Phi-3-mini and Qwen2.5), and Claude, a larger one that runs on Anthropic's computers.
Fine-tuning / Training	Teaching one of the small AI models to get better at banking questions specifically, by showing it thousands of example questions and good answers.
Colab	A free website from Google that lends you a powerful computer (with a graphics card, or "GPU") for a little while, which is what training a model needs.
Dashboard	A web page that shows you, in charts and tables, how well the system is doing.

If a word appears that is not in this table, this manual will explain it the first time it is used.

1. What you will end up with

By the end of this manual, on your own computer, you will have:

1. A folder called `two-tier-system` containing all the project's code.
2. Two small AI models (called Phi-3-mini and Qwen2.5) running privately on your own computer through a program called Ollama — one specialised for payments questions, one for retail banking questions.
3. A connection to Claude (Anthropic's AI) for the harder questions the small models are not confident about.
4. A generator that creates thousands of realistic, made-up practice questions and answers to test the system with.
5. (Optional, and the most advanced part) Your own custom-trained versions of the two small models, trained for free using Google's computers.
6. A dashboard — a web page — that shows how the whole system is performing.

Everything runs on your own computer except the training step (which borrows Google's computer for a couple of hours) and the harder questions (which are sent to Claude).

2. Before you start — what you need

- A Windows or Mac computer. This manual gives separate steps for each, clearly marked  **Windows** and  **Mac**.
- An internet connection.
- About 10 GB of free storage space.
- An email address, to create a couple of free accounts (GitHub, Google, Anthropic).
- Patience — some steps, like downloading an AI model, take several minutes. That is normal, not a mistake.

A note on cost: everything in this manual is free, except the Claude questions (a few cents in total for normal testing) and, optionally, a very small amount if you choose to use a faster Google Colab computer instead of the free one.

3. Install the basic tools

You need three programs before anything else: **Ollama** (runs the small AI models), **Python** (runs our project's code), and **Git** (downloads and manages the project's files). Install all three now, in this order.

3.1 Install Ollama

Ollama is the program that runs the small AI models directly on your computer, privately, with no internet needed once a model is downloaded.

Windows

1. Open your web browser and go to **ollama.com/download**.
2. Click the **Download for Windows** button.
3. When the file finishes downloading, double-click it (it will be named something like `OllamaSetup.exe`).
4. Click **Install** and wait for it to finish. You do not need to change any options.
5. Ollama is now running quietly in the background. You will see a small llama icon appear near the clock, bottom-right of your screen.

Mac

1. Open your web browser and go to **ollama.com/download**.
2. Click the **Download for macOS** button.
3. Open the downloaded file (it will be named something like `Ollama.dmg`).
4. Drag the Ollama icon into your Applications folder, as the window instructs.
5. Open **Ollama** from your Applications folder. Click **Open** if a security warning appears (this is normal for a program downloaded from the internet).
6. Ollama is now running quietly in the background. You will see a small llama icon appear near the clock, top-right of your screen.

Check it worked (both Windows and Mac)

Open a terminal: - **Windows**: click the Start menu, type `PowerShell` , and press Enter. - **Mac**: press `Cmd + Space` , type `Terminal` , and press Enter.

Type this command and press Enter:

```
ollama --version
```

What you should see: a line showing a version number, like `ollama version 0.34.2` . If instead you see an error saying the command is not recognised, close the terminal, reopen it, and try again — Ollama sometimes needs the terminal to be reopened once after installing.

3.2 Install Python

Python is the programming language our project's code is written in.



Windows

1. Go to python.org/downloads in your browser.
2. Click the big yellow **Download Python** button (it will show the latest version number, such as 3.12).
3. Open the downloaded file.
4. **Important:** on the very first screen of the installer, tick the box at the bottom that says "**Add python.exe to PATH**". This step is easy to miss and causes problems later if skipped.
5. Click **Install Now** and wait for it to finish.



Mac

Most Macs already have a version of Python built in, but it is usually too old for us. Install a current one: 1. Go to python.org/downloads in your browser. 2. Click the download button for macOS. 3. Open the downloaded file and follow the installer's prompts, clicking **Continue** and **Install** on each screen.

Check it worked (both)

In your terminal, type:

```
python --version
```

What you should see: something like `Python 3.12.4` . On some Macs, you may need to type `python3` instead of `python` — if `python --version` gives an error, try `python3 --version` instead, and use `python3` in place of `python` for every command in the rest of this manual.

3.3 Install Git

Git is the tool that downloads and keeps track of the project's files.



Windows

1. Go to git-scm.com/downloads in your browser.

2. Click **Windows**, and the download will start automatically.
3. Open the downloaded file and click **Next** on every screen, then **Install**. The default options are all fine.

Mac

1. Open your terminal.
2. Type `git --version` and press Enter.
3. If Git is not already installed, your Mac will pop up a window offering to install "Command Line Developer Tools." Click **Install** and wait a few minutes for it to finish.

Check it worked (both)

In your terminal, type:

```
git --version
```

What you should see: something like `git version 2.46.0` .

4. Get the project files onto your computer

There are two situations. Find the one that matches you and follow only that one.

- **Situation A:** you already have this project's code on GitHub, and you want it on a computer for the first time. Follow **4.1**.
- **Situation B:** you are starting completely fresh and have never put this project on GitHub. Follow **4.2** first, then come back and do **4.1** on any other computer later.

4.1 Download ("clone") the project from GitHub

This is how anyone — including you, on a new computer — gets a full copy of the project.

1. Open your terminal.
2. Decide where you want the project folder to live, for example your Desktop. Move into that location. For example, to use your Desktop:
3.  **Windows:** `cd Desktop`
4.  **Mac:** `cd Desktop`
5. Type this command, replacing `YOUR-USERNAME` and `YOUR-REPO-NAME` with the real GitHub username and repository name (ask the project owner for these if you do not know them): `git clone https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git`
6. Press Enter. You will see text scroll by as Git downloads every file.
7. **What you should see when it finishes:** a line like `Resolving deltas: 100% done` , and then the terminal returns to a normal prompt with no error.
8. Move into the new folder it created: `cd YOUR-REPO-NAME` (If the project is called `two-tier-system` , this would be `cd two-tier-system` .)

If the repository is **private** (not visible to the public), plain `git clone` will ask for a username and password, and a plain password will not work. Skip ahead to section 4.3 below, which explains how to make a "token" that works instead.

4.2 Create the GitHub repository for the first time

Only do this once, right at the very start of the project.

1. Go to **github.com** and either log in or click **Sign up** to create a free account.
2. Once logged in, click the **+** icon in the top-right corner, then **New repository**.
3. Give it a name, for example `two-tier-system`.
4. Choose **Private** (recommended, so only people you choose can see it) or **Public**.
5. **Leave every checkbox unticked** — do not add a README, .gitignore, or license from this screen.
6. Click **Create repository**.
7. On your computer, open your terminal and go to the folder that has your project files in it.
8. Type these commands one at a time, pressing Enter after each: `git init` `git add .` `git commit -m "Initial commit"` `git branch -M main` `git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git` `git push -u origin main`
9. The first time you push, it will ask you to log in. See section 4.3 below for how.

4.3 Logging in to GitHub from your terminal (Personal Access Token)

GitHub no longer accepts your normal account password when your terminal asks for one. Instead, you create a special code called a "token" and use that instead.

1. In your browser, go to **github.com**, log in, then click your profile picture (top-right) → **Settings**.
2. Scroll all the way down the left-hand menu and click **Developer settings**.
3. Click **Personal access tokens** → **Tokens (classic)**.
4. Click **Generate new token** → **Generate new token (classic)**.
5. Give it a name you will recognise, like `my-laptop`.
6. Tick the box next to **repo**.
7. Scroll down and click **Generate token**.
8. **Copy the long code shown immediately** — it looks like `ghp_` followed by letters and numbers. You will not be able to see it again after leaving this page. If you lose it, just generate a new one.
9. Back in your terminal, whenever Git asks for a **username**, type your GitHub username. Whenever it asks for a **password**, paste this token instead of your real password.

Tip: your terminal will usually remember this after the first time, so you should not need to paste it every single time.

5. Set up the project's Python environment

This creates a private, self-contained box for this project's software, so it does not interfere with anything else on your computer.

1. Open your terminal and move into the project folder (skip this if you are already there): `cd two-tier-system`
2. Create the virtual environment: `python -m venv venv` This takes a few seconds and creates a new folder called `venv` inside your project. You will not need to look inside it.
3. Turn it on ("activate" it):
4. 🪟 **Windows (PowerShell):** `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned .\venv\Scripts\Activate.ps1` (The first line is a one-time permission fix. If it asks you a yes/no question, type `Y` and press Enter.)
5. 🍏 **Mac (Terminal):** `source venv/bin/activate`
6. **What you should see:** the very start of your terminal's input line now shows `(venv)` in front of everything else. From now on, every single time you open a new terminal window to work on this project, you must repeat this activation step first, or nothing will work correctly. You do not need to recreate the `venv` — just reactivate it.
7. Install all the software this project needs, in one go: `pip install -r requirements.txt` This will take a minute or two and show a lot of scrolling text. That is normal.

How to know it worked: type `pip list` and press Enter. You should see a long list of names including `pandas`, `requests`, and `anthropic`.

6. Get a Claude (Anthropic) API key

The harder questions in this project are sent to Claude, an AI made by a company called Anthropic. To use it from our code, you need your own private key.

1. Go to **console.anthropic.com** in your browser (this is different from `claude.ai`, which is for chatting — this "console" address is for developers).
2. Sign up or log in.
3. Click **Settings**, then **Billing**, and add a payment card. This is a pay-as-you-go service — for this project's normal use, the total cost is typically only a few cents.
4. Click **API Keys** in the left-hand menu.
5. Click **Create Key**, give it a name such as `two-tier-system`, and click **Create**.
6. **Copy the key immediately.** It starts with `sk-ant-` and will not be shown to you again after you leave the page. If you lose it, delete it and create a new one.
7. Now save this key onto your computer so the project can use it, without ever typing it into any file that gets shared or uploaded:
8. 🪟 **Windows (PowerShell):** `setx ANTHROPIC_API_KEY "paste-your-real-key-here"` After running this, **close your terminal window completely and open a brand new one** — this step only takes effect in new windows, and you must reactivate the `venv` (step 3 above) in the new window.
9. 🍏 **Mac (Terminal):** `echo 'export ANTHROPIC_API_KEY="paste-your-real-key-here"' >> ~/.zshrc`
`source ~/.zshrc`

How to know it worked: - 🪟 **Windows:** type `echo %env:ANTHROPIC_API_KEY%` - 🍏 **Mac:** type `echo $ANTHROPIC_API_KEY`

Either way, it should print your key back to you. If it prints nothing, repeat the step above carefully.

Keep this key private. Never paste it into a chat message, an email, or a file that you upload or push to GitHub — anyone who has it can spend money on your account.

7. Download the two AI models

The project uses two small AI models that run privately on your computer: **Phi-3-mini** (for payments questions) and **Qwen2.5** (for retail banking questions).

Make sure Ollama is running (you should see its llama icon near your clock) and your terminal has `(venv)` showing. Then type each of these, one at a time, pressing Enter and waiting for each to finish before starting the next:

```
ollama pull phi3:mini
```

```
ollama pull qwen2.5:1.5b
```

Each one shows a progress bar and downloads a few gigabytes, so this can take several minutes depending on your internet speed.

How to know it worked: type `ollama list` and press Enter. You should see both `phi3:mini` and `qwen2.5:1.5b` listed, each with a size next to it.

8. Create the practice (synthetic) data

Before testing the system, we generate thousands of realistic, made-up banking questions and answers to test it with. Nothing here is real customer data — it is all invented by a small program, specifically so it is safe to use and share.

In your terminal, with `(venv)` showing, move to the project folder and run:

```
python scripts/fetch_data.py
```

This downloads a public dataset of real bank customer-service questions (with no real customer information) and a public dataset of financial question-and-answer pairs.

```
python scripts/fetch_bitext_banking.py
```

This downloads a larger public dataset of realistic banking conversations and automatically sorts it into "payments" and "retail banking" questions.

```
python scripts/generate_synthetic_bitext.py
```


This is the one that invents new practice data. **What you should see:** ten lines, each saying something like `Wrote data/synthetic/synthetic_bitext_01.csv: 1600 rows`, followed by a final summary line. This produces 10 files with at least 1,500 made-up question-and-answer pairs each, including both everyday questions and unusual, tricky situations (like fraud or a family bereavement), so the system can be tested on both simple and difficult cases.

How to know it worked: -  **Windows:** `dir data\synthetic` -  **Mac:** `ls data/synthetic`

You should see 10 files named `synthetic_bitext_01.csv` through `synthetic_bitext_10.csv`, plus one file called `synthetic_manifest.csv`.

9. Run the system for the first time

This sends a batch of practice questions through the whole system: the local models answer first, and anything they are not confident about is sent to Claude.

In your terminal, with `(venv)` showing and Ollama running, type:

```
python -m src.pipeline 10
```

The number `10` means "use 10 random practice questions" — a small number, good for a first try. Once everything is working, you can run larger batches, for example `python -m src.pipeline 100`.

What you should see: the terminal prints one block of text per question, showing which model answered it, how confident it was, and whether it was answered locally or sent to Claude. At the end, it prints a summary count and saves the results.

How to know it worked: -  **Windows:** `dir results\logs` -  **Mac:** `ls results/logs`

You should see files including `results_log_combined.csv` and `results_history.csv`.

10. Set up and open the dashboard

The dashboard is a web page that shows charts and tables of how the system is performing.

1. In your terminal, with `(venv)` showing, install the dashboard's small extra requirements (only needed once): `pip install -r dashboard/backend/requirements.txt`
2. Start the dashboard: `python -m uvicorn dashboard.backend.main:app --port 8000`
3. **What you should see:** a few lines of text ending with something like `Uvicorn running on http://0.0.0.0:8000`. Leave this terminal window open — closing it turns the dashboard off.
4. Open your web browser and go to: `http://localhost:8000`
5. You should see the dashboard appear, showing your recent runs, confidence charts, and tables.

To stop the dashboard: click back into that terminal window and press `Ctrl + C`.

To use the dashboard again later: repeat steps 2 to 4 (you do not need to reinstall anything).

11. (Optional, advanced) Train your own custom versions of the models

This is the most advanced part of this manual. It teaches the two small models to be better at banking questions specifically, using real examples. It is optional — the system works without it — but it produces noticeably better answers.

This step needs a more powerful computer than most laptops have, so instead we borrow one for free from Google, using a website called **Google Colab**. This part takes roughly 2 to 3 hours of waiting per model (the computer does the work — you do not need to watch it the whole time), plus about 15 to 20 minutes of your own active steps.

11.1 What you need before starting

- A free Google account (the same kind used for Gmail).
- Your project already on GitHub (see Section 4).
- Your GitHub Personal Access Token from Section 4.3.

11.2 Open Google Colab and turn on a free graphics card

1. Go to **colab.research.google.com** and log in with your Google account.
2. Click **New notebook**.
3. Click **Runtime** in the top menu, then **Change runtime type**.
4. Under "Hardware accelerator," choose **T4 GPU**, then click **Save**.

11.3 Connect Colab to your Google Drive

This makes sure your trained model is saved safely, even if Colab disconnects — Colab's own storage is temporary and gets wiped, but your Google Drive is permanent.

Click the **+ Code** button to add a new block ("cell"), then type or paste this and press the Play button (▶) on the left of it, or press Shift+Enter:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up window will ask you to log in to Google and allow access. Click through it and approve it.

What you should see: a message saying `Mounted at /content/drive`.

11.4 Store your GitHub token safely in Colab

Rather than typing your secret GitHub token directly into a notebook cell (where it could accidentally be saved or shared), store it in Colab's built-in, private "Secrets" feature.

1. On the left-hand side of the Colab screen, click the **key-shaped icon** ("Secrets").
2. Click **+ Add new secret**.
3. For the name, type `GITHUB_TOKEN`.
4. For the value, paste your GitHub Personal Access Token from Section 4.3.
5. Turn on the toggle switch labelled **Notebook access**.

11.5 Download ("clone") your project into Colab

Add a new cell and run:

```
from google.colab import userdata
token = userdata.get('GITHUB_TOKEN')

%cd /content/drive/MyDrive
!git clone https://{token}@github.com/YOUR-USERNAME/YOUR-REPO-NAME.git
%cd YOUR-REPO-NAME
```

Replace `YOUR-USERNAME` and `YOUR-REPO-NAME` with your real details. **What you should see:** text ending in something like `Resolving deltas: 100% done`.

By putting this inside your Google Drive folder (`/content/drive/MyDrive`) rather than Colab's own temporary storage, your project's files will still be there even if Colab disconnects or you switch to a different type of graphics card later.

11.6 Install the training software

```
!pip install -r training/requirements.txt
```

This takes a couple of minutes and shows a lot of scrolling text, which is normal.

11.7 Prepare the training data

```
!python scripts/fetch_bitext_banking.py
```

This downloads and sorts the real example conversations the models will learn from, the same way it did on your own computer in Section 8.

11.8 Train the Payment Assistant (Phi-3-mini)

```
!python training/train_payments.py
```

This is the step that takes the longest — roughly 2 to 2.5 hours on the free graphics card. You will see a stream of numbers scrolling by (this is normal, technical progress information). You do not need to understand every line; just watch for it to finish.

What you should see when it finishes: a message saying training is complete and that the finished model has been saved directly to your Google Drive, inside a folder called `LJMU_Research/two-tier-system-models` (or wherever you have your project set up) — you do not need to manually download anything.

Important: keep the Colab browser tab open and your computer awake for this whole time. If your computer goes to sleep or the tab is closed, training will stop partway through and you will need to start again.

11.9 Start a fresh session, then train the Retail Bank Assistant (Qwen2.5)

Training a second model in the same session can run out of memory, so start clean:

1. Click **Runtime** → **Restart runtime**.
2. Repeat steps 11.3 (mount Drive), 11.5 (clone the project again), 11.6 (install requirements), and 11.7 (prepare data) — a runtime restart clears everything, so these need to be done again.
3. Then run: `python !python training/train_retail.py`
4. Wait for it to finish, the same way as before — this one also takes roughly 2 to 2.5 hours.

11.10 Get the finished models onto your own computer

The two trained model files are now sitting safely in your Google Drive, inside a folder that training created, for example `LJMU_Research/two-tier-system-models`. The easiest way to get them onto your own computer is to install **Google Drive for Desktop**, which makes your Google Drive appear as a normal folder on your computer:

1. Go to google.com/drive/download in your browser.
2. Download and install "Drive for desktop" for your operating system (Windows or Mac), following the installer's prompts.
3. Sign in with the same Google account you used for Colab.
4. When asked to choose a sync option, choose **"Mirror files"**, so a real, permanent copy is kept on your own computer.
5. Wait for it to finish downloading your files — this can take a little while for large files. You will see a small syncing icon (spinning circle) near your clock while it works, which turns into a plain checkmark once it is done. Do not move on to the next step until it shows the checkmark.
6. Once finished, you will be able to find your trained models on your own computer, inside your Google Drive folder, at the path you saved them to.

Tip: if you would rather not install anything extra, you can instead click the folder icon on the left side of the Colab screen, find your model files, and right-click → **Download** — but for large files, Google Drive for Desktop is far more reliable.

12. Bring your trained models into Ollama

Now that the trained model files are on your own computer, you need to tell Ollama about them.

1. Open your terminal (not the venv one specifically — this part does not need it) and move to the folder containing your trained Payment Assistant model file (the exact path depends on where your Google Drive folder is on your computer, for example):
2.  **Windows:** `cd "G:\My Drive\LJMU_Research\two-tier-system-models\payment_assistant_gguf"`
3.  **Mac:** `cd "/Users/YOUR-NAME/Google Drive/My Drive/LJMU_Research/two-tier-system-models/payment_assistant_gguf"`
4. Check there is a file ending in `.gguf` in this folder:
5.  **Windows:** `dir`

6. 🍎 **Mac:** `ls`

7. There should already be a small file called `Modelfile` in the same folder (the training step creates it automatically). If there is not, create one yourself:

8. Open a plain text editor (Notepad on Windows, TextEdit on Mac).

9. Type exactly this one line (matching your actual `.gguf` file's name): `FROM ./payment_assistant.gguf`

10. Save the file with the exact name `Modelfile`, with no file extension like `.txt` (on Windows, when saving, change "Save as type" to "All Files").

11. Now create the Ollama model: `ollama create payment-assistant -f Modelfile`

12. Repeat the same steps (1 to 4) for the Retail Bank Assistant, in its own folder, using this command instead:

```
ollama create retail-bank-assistant -f Modelfile
```

How to know it worked: type `ollama list`. You should now see `payment-assistant` and `retail-bank-assistant` in the list, alongside the original `phi3:mini` and `qwen2.5:1.5b`.

Try one out directly:

```
ollama run payment-assistant "Why was my card payment declined?"
```

You should see a detailed, written-out answer appear.

12.1 Tell the project to use your new trained models

1. Open the project folder in a plain text editor and open the file `src/config.py`.

2. Find the section that looks like this: `python "payments": { "model": "phi3:mini", ... },`
`"retail_bank": { "model": "qwen2.5:1.5b", ... },`

3. Change `"phi3:mini"` to `"payment-assistant"`, and `"qwen2.5:1.5b"` to `"retail-bank-assistant"`.

4. Save the file.

5. Run the system again to try your newly trained models: `python -m src.pipeline 10`

13. Everyday commands, once everything is set up

After finishing this manual once, here is everything you need for normal day-to-day use. Always start by opening your terminal, moving into the project folder, and turning on the venv:

```
cd two-tier-system
```

- 🖥️ **Windows:** `.\venv\Scripts\Activate.ps1`
- 🍎 **Mac:** `source venv/bin/activate`

Then, whenever you want to:

To do this	Type this
Run the system on 10 practice questions	<code>python -m src.pipeline 10</code>
Run the system on 100 practice questions	<code>python -m src.pipeline 100</code>
Open the dashboard	<code>python -m uvicorn dashboard.backend.main:app --port 8000</code> , then open <code>http://localhost:8000</code> in your browser
Save your changes to GitHub	<code>git add .</code> then <code>git commit -m "describe what changed"</code> then <code>git push</code>
Get someone else's latest changes from GitHub	<code>git pull</code>
See which AI models Ollama has	<code>ollama list</code>

14. If something goes wrong

What you see	What it means, and what to do
<code>'ollama' is not recognized or command not found: ollama</code>	Ollama is not installed, or your terminal needs reopening after installing it. Reopen your terminal and try again.
<code>No module named uvicorn</code> (or any other module)	Your venv is not turned on. Look for <code>(venv)</code> at the start of your terminal line — if it is missing, go back to Section 5, step 3.
Git asks for a password and rejects it	GitHub no longer accepts plain passwords. Use your Personal Access Token instead — see Section 4.3.
<code>Connection refused</code> when opening the dashboard	The dashboard is not running. Go back to Section 10 and run the <code>uvicorn</code> command again, and keep that terminal window open.
The Claude questions fail with an authorization error	Your API key is not set correctly. Go back to Section 6 and check it, remembering to open a brand-new terminal window afterwards.
Colab training stops partway through	Your browser tab was closed, your computer slept, or the free session simply timed out. Reopen Colab and start again from Section 11.5 — this is a normal thing to happen occasionally, not a mistake you made.
A file you edited does not seem to have saved	Some editors add a hidden <code>.txt</code> at the end of file names. Check the exact file name carefully, and when saving, choose "All Files" instead of a specific file type.

If you hit an error not listed here, copy the exact error message somewhere safe before doing anything else — it usually says exactly what went wrong.